📘 Project Report

🎯 Guess The Number PRO Game (Python Tkinter)

---

1. Introduction

Guess The Number PRO is a desktop-based game developed using **Python** and **Tkinter**. The main objective of the game is for the player to guess a randomly generated number within a limited time and minimum attempts.

The game includes multiple difficulty levels, sound effects, and a leaderboard system, making it interactive and user-friendly.

2. Objectives of the Project

The main objectives of this project are:

- To develop an interactive number-guessing game.
- To improve logical thinking and decision-making skills.
- To implement GUI-based programming in Python.
- To store and display player scores using files.
- To provide multiple difficulty levels.
- To include sound feedback for user actions.

3. Technologies Used

Technology	Purpose
Python	Main programming language
Tkinter	Graphical User Interface (GUI)
Random Module	Generate random numbers
OS Module	File handling
Winsound	Sound effects (Windows)
Text File	Store leaderboard data

4. System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 2GB RAM
- Keyboard and Mouse

Software Requirements

- Windows Operating System
- Python 3.x
- Tkinter Library (Default with Python)
- Text Editor (VS Code / IDLE)

5. Features of the Project

✅ Multiple Difficulty Levels

The game provides three levels:

Level	Range	Time
Easy	1 – 100	60 sec
Medium	1 – 500	45 sec
Hard	1 – 1000	30 sec

✅ Timer System

- A countdown timer is shown on the screen.
- If time runs out, the game ends automatically.

✅ Leaderboard System

- Stores player name, attempts, and level.
- Saved in a file named leaderboard.txt.
- Displays Top 5 scores.

✅ **Sound Effects**

Different sounds for:

- Win
- Lose
- Button Click

✅ **Graphical User Interface**

- Attractive background
- Buttons for level selection
- Input box for guessing
- Result display
- Score display

6. Working of the System

Step 1: Start Game

- Player enters their name.
- Selects difficulty level.
- Game starts automatically.

Step 2: Number Generation

- Computer generates a random number based on selected level.

Step 3: User Input

- Player enters a number.
- Clicks the **Guess** button.

Step 4: Result Checking

- If guess is correct → Win.
- If guess is high → "Too High".
- If guess is low → "Too Low".

Step 5: Timer Control

- Timer decreases every second.
- If timer reaches 0 → Game Over.

Step 6: Score Saving

- On winning, score is saved.
- Leaderboard is updated.

7. Program Structure

Important Functions

Function Name	Purpose
start_game()	Starts new game
check_guess()	Checks user input
start_timer()	Controls countdown
game_over()	Ends game
save_score()	Stores data
load_leaderboard()	Reads data
change_level()	Changes difficulty

File Used

- leaderboard.txt → Stores player scores.

8. Advantages of the Project

✓ Easy to use
✓ Interactive GUI
✓ Improves logical skills
✓ Motivating with leaderboard
✓ Lightweight application
✓ Beginner-friendly code

9. Limitations of the Project

❌ Works only on Windows (winsound)
❌ No online multiplayer
❌ Limited graphics
❌ No mobile version
❌ No database (uses text file only)

10. Future Enhancements

In future, this project can be improved by:

- Adding online multiplayer mode
- Using database instead of text file
- Adding more levels
- Adding animations
- Making cross-platform version
- Adding user login system
- Adding dark/light themes

11. Conclusion

The **Guess The Number PRO** project is a simple yet effective GUI-based game developed using Python and Tkinter.
It successfully demonstrates:

- File handling
- GUI programming
- Event handling
- Timer management
- Game logic

CODE

```
import tkinter as tk
import random
import os
import winsound   # Works on Windows only

# ---------------- FILES ----------------
LEADERBOARD_FILE = "leaderboard.txt"

# ---------------- GAME SETTINGS ----------------
LEVELS = {
    "Easy": {"max": 100, "time": 60},
    "Medium": {"max": 500, "time": 45},
    "Hard": {"max": 1000, "time": 30}
}

current_level = "Easy"
computer_value = 0
count = 0
time_left = 0
timer_running = False

# ---------------- SOUND ----------------
def play_sound(type):

    if type == "win":
        winsound.MessageBeep(winsound.MB_ICONASTERISK)

    elif type == "lose":
```

```
        winsound.MessageBeep(winsound.MB_ICONHAND)

    elif type == "click":
        winsound.MessageBeep(winsound.MB_OK)

# ---------------- LEADERBOARD ----------------
def load_leaderboard():

    scores = []

    if os.path.exists(LEADERBOARD_FILE):

        with open(LEADERBOARD_FILE, "r") as f:

            for line in f:
                name, attempts, level = line.strip().split(",")
                scores.append((name, int(attempts), level))

    return scores

def save_score(name, attempts, level):

    with open(LEADERBOARD_FILE, "a") as f:
        f.write(f"{name},{attempts},{level}\n")

def show_leaderboard():

    board = load_leaderboard()

    board.sort(key=lambda x: x[1])
```

```
    text = "🏆 Leaderboard (Top 5)\n\n"

    for i, score in enumerate(board[:5], 1):
        text += f"{i}. {score[0]} - {score[1]} tries ({score[2]})\n"

    leaderboard_label.config(text=text)

# ---------------- TIMER ----------------
def start_timer():

    global time_left, timer_running

    if not timer_running:
        return

    if time_left > 0:

        timer_label.config(text=f"⏱ Time: {time_left}s")
        time_left -= 1

        root.after(1000, start_timer)

    else:
        game_over()

# ---------------- GAME OVER ----------------
def game_over():

    global timer_running
```

```
    timer_running = False

    result_label.config(
        text=f"⏰ Time Up! Number was: {computer_value}",
        fg="red"
    )

    play_sound("lose")

    guess_btn.config(state="disabled")

# ---------------- GAME ----------------
def start_game():

    global computer_value, count, time_left, timer_running

    play_sound("click")

    settings = LEVELS[current_level]

    computer_value = random.randint(1, settings["max"])
    time_left = settings["time"]
    count = 0

    timer_running = True

    entry.delete(0, tk.END)

    range_label.config(text=f"Range: 1 - {settings['max']}")
```

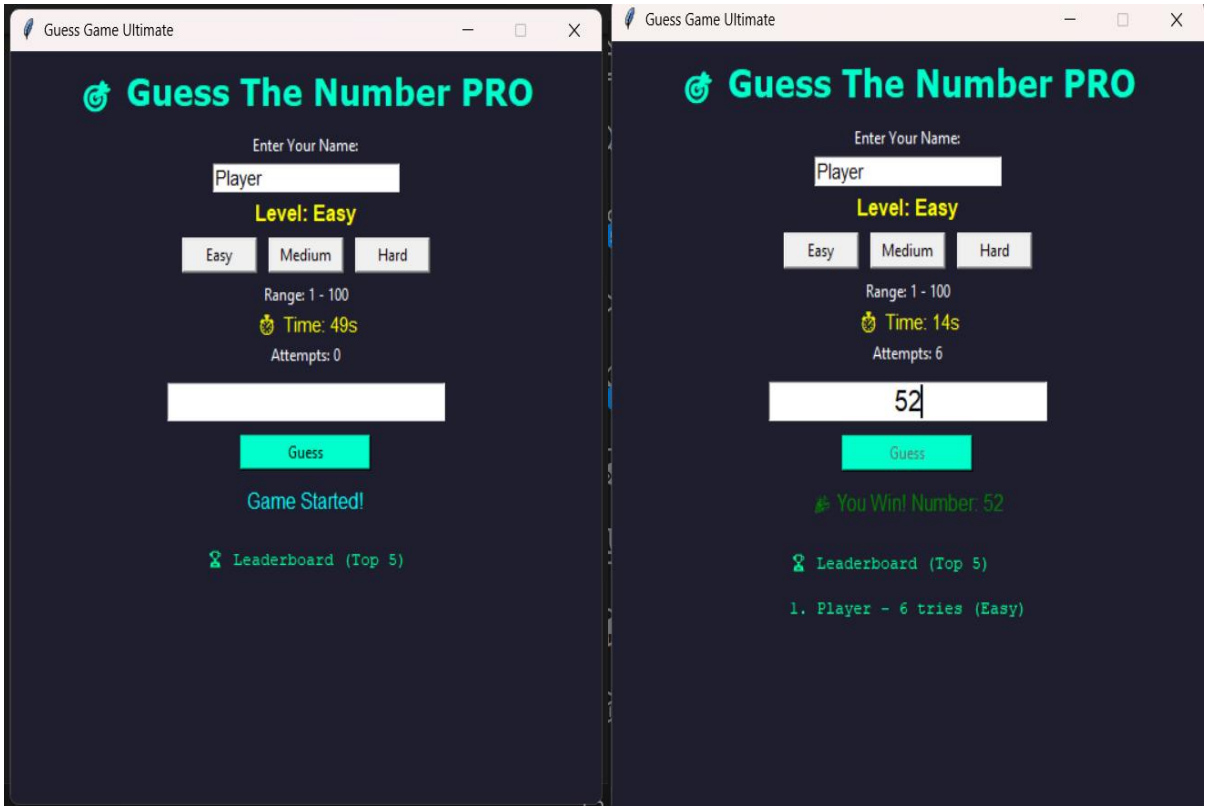
```
    attempts_label.config(text="Attempts: 0")
    result_label.config(text="Game Started!", fg="cyan")

    guess_btn.config(state="normal")

    start_timer()

def check_guess():

    global count, timer_running

    play_sound("click")

    try:

        user = int(entry.get())
        count += 1

        attempts_label.config(text=f"Attempts: {count}")

        if user == computer_value:

            timer_running = False

            result_label.config(
                text=f"🎉 You Win! Number: {computer_value}",
                fg="green"
            )
```

```
            play_sound("win")

            guess_btn.config(state="disabled")

            save_score(name_entry.get(), count, current_level)

            show_leaderboard()

        elif user > computer_value:
            result_label.config(text="⬆️ Too High!", fg="orange")

        else:
            result_label.config(text="⬇️ Too Low!", fg="orange")

    except:
        result_label.config(text="⚠️ Enter Number Only!", fg="red")

# ---------------- LEVEL ----------------
def change_level(level):

    global current_level

    current_level = level

    level_label.config(text=f"Level: {level}")

    start_game()

# ---------------- UI ----------------
root = tk.Tk()
```

```
root.title("Guess Game Ultimate")
root.geometry("520x550")
root.configure(bg="#1e1e2f")
root.resizable(False, False)

# Title
tk.Label(
    root,
    text="🎯 Guess The Number PRO",
    font=("Verdana", 20, "bold"),
    fg="#00ffcc",
    bg="#1e1e2f"
).pack(pady=10)

# Name
tk.Label(root, text="Enter Your Name:", bg="#1e1e2f", fg="white").pack()

name_entry = tk.Entry(root, font=("Arial", 11))
name_entry.pack(pady=3)
name_entry.insert(0, "Player")

# Level
level_label = tk.Label(
    root,
    text="Level: Easy",
    bg="#1e1e2f",
    fg="yellow",
    font=("Arial", 12, "bold")
)
level_label.pack()
```

```
level_frame = tk.Frame(root, bg="#1e1e2f")
level_frame.pack(pady=5)

for lvl in LEVELS:
    tk.Button(
        level_frame,
        text=lvl,
        width=8,
        command=lambda l=lvl: change_level(l)
    ).pack(side="left", padx=5)

# Info
range_label = tk.Label(root, fg="white", bg="#1e1e2f")
range_label.pack()

timer_label = tk.Label(root, fg="yellow", bg="#1e1e2f", font=("Arial", 11))
timer_label.pack()

attempts_label = tk.Label(root, fg="white", bg="#1e1e2f")
attempts_label.pack()

# Input
entry = tk.Entry(root, font=("Arial", 16), justify="center")
entry.pack(pady=10)

# Button
guess_btn = tk.Button(
    root,
    text="Guess",
```

```
    width=15,
    bg="#00ffcc",
    command=check_guess
)
guess_btn.pack()

# Result
result_label = tk.Label(
    root,
    text="Choose Level to Start",
    fg="cyan",
    bg="#1e1e2f",
    font=("Arial", 12)
)
result_label.pack(pady=10)

# Leaderboard
leaderboard_label = tk.Label(
    root,
    text="🏆 Leaderboard",
    bg="#1e1e2f",
    fg="#00ff99",
    font=("Courier", 10),
    justify="left"
)
leaderboard_label.pack(pady=10)

show_leaderboard()

# Start default
```

```
change_level("Easy")

root.mainloop()
```

PICTURE